

Three Ways to Ground Genie Agent

We asked two executive questions, three times each, to three Genie Agent setups on the same catalog: raw tables, then a semantic layer, then a semantic layer with an ontology and a knowledge graph. Same product every time. We only changed what we gave it to work with.

QUESTIONS	SETUPS
2	3
CHECKPOINTS SCORED	SESSIONS CAPTURED
15 per setup	6
CATALOG UNDER TEST	
gold_dev_analytics	

THE FINDING

Once we lock down what the agent can reach, two problems are left: it uses the wrong word, or it gives an incomplete answer. Certified measures fix the first. Nothing about them fixes the second.

All three setups stayed inside the catalog. None invented a table, made up a supplier, or claimed data was missing when it was there. That worst kind of failure cannot happen in a bounded Genie Agent space. What is left is quieter and much harder to spot.

The bottom two setups scored 27% and 40% — one checkpoint apart — and failed at different things. Both told us which warehouses and parts a supplier outage would hit, and both left out a quarter of the sourcing graph. One of the three locations about to run out of stock was in the part they missed. Neither warned us anything was missing. Setup 3 scored 87%: the only one that gave us the full exposure list, the date the stock figures came from, and advice covering every at-risk location.

Setup 3 also lost one checkpoint outright to the semantic layer. Setup 2 put a price on the exposure — ₹53.89 Lakh, broken down by part and by warehouse. Setup 3 reported units and never turned them into money. We built the graph for navigation only, and on this question that choice cost us.

VERIFIED ACCURACY ACROSS BOTH QUESTIONS · 15 CHECKPOINTS EACH

1 Agent, tables only		27%
----------------------	--	-----

4 of 15

2 Agent, semantic layer

40%

6 of 15

3 Agent + ontology + KG

87%

13 of 15

Each rung scores higher than the one below, but the steps are not the same size. The semantic layer adds **13 points**, all of it from picking the right word on Question 1 and pricing the answer on Question 2. It adds nothing to completeness. The ontology and graph add **47 points**, almost all of it completeness. Two different problems, two different layers. Neither does the other's job.

01 What actually changed between the three

All three are published Genie Agent spaces on `gold_dev_analytics`, and none can read a table we did not hand it. Two things change across the rungs: whether we certified the measures, and whether we declared the relationships.

	1 · TABLES ONLY	2 · + SEMANTIC LAYER	3 · + ONTOLOGY & KG
Data sources	15 dims + 5 facts (20)	+ 5 metric views (25)	the same 25
Descriptions & synonyms	None	None	Every table, every column; 3 synonyms per key column
Text instructions	None	1 block — routing, clarification, empty-period, formatting	The same block + generated ontology grounding
Certified snippets	None	None	8 measures + 4 filters
SQL functions	None	None	<code>kg_find_node</code> , <code>kg_neighbors</code>
Sourcing relation reachable	✗ <code>dim_supplier_contract</code> not exposed	✗ <code>dim_supplier_contract</code> not exposed	✓ as typed edges in <code>kg_edges</code>
Typical failure	Answers one reading of the question, then says no other reading existed	Correct maths over an incomplete list, every time	Answers narrowly, but says what it measured

The semantic layer bought us nothing on the relationship question. Five certified metric views are the only difference between Setup 1 and Setup 2. Setup 2 still came back with the same incomplete part list, using about twice as many queries to get there. A semantic layer certifies how we calculate a number along a path that already exists. It cannot build a new path, and it cannot tell us we took the wrong one.

What Setup 3 adds is the ontology, and the ontology is what reaches the relationship. Neither space exposes `supply_chain_analytics.dim_supplier_contract`, the table that actually records who supplies what — it is not one of the 15 dimensions, and no query either space writes can touch it. Setup 3 has instead a declared relationship, `mfg:supplierSuppliesPart`, materialised into `kg_edges` and read through two functions. Those edges were built from `dim_supplier_contract` at materialisation time, so Setup 3 does reach a fact Setup 2 cannot — but only because we said in advance that the relationship exists and was worth carrying. That is the whole difference: not a bigger catalog, a declared one.

02 Question 1 — an ambiguous word

Q1 “What did we spend last month?”

The maths is easy, and that is the point. Our data holds at least three unrelated things a C-level user might call “spend”: campaign media spend, cost of poor quality, and discount given away. The ontology declares four meanings — `ea:Cost` disambiguatesTo `ea:CampaignSpend`, `ea:CostPerLead`, `ea:COPQ`, `ea:Discount` — and the numbers are nowhere near each other in size. The agent can calculate any of them perfectly and still hand us the wrong answer. “Last month” hides a second trap: our domains do not end on the same date, so the agent has to work it out per domain rather than read it off a calendar.

1 • Tables only

15 dims + 5 facts

1/6

ANSWERED ONE OF THREE

2 • Semantic layer

+ 5 metric views, instructions

3/6

ASKED, BUT SILENTLY

3 • Ontology + KG

+ TTL grounding, snippets, UDFs

5/6

ASKED, AND SHOWED ITS WORKING

1 • Tables only

What did we spend last month? Reviewable

Chat Monitor Benchmark

Configure Share

What did we spend last month?

> Thinking complete

Total marketing spend in August 2026 (last month): ₹44,000,000 (₹4.4 crore) 1

This came from two campaigns that started in August:

- Independence Day Bonanza — ₹32,000,000 (Aug 1–31)
- Ertiga Family Festival — ₹12,000,000 (Aug 1–Oct 31)

2

Note: this reflects marketing campaign spend, the only “spend” metric available in the data; August is also the most recent month with campaign records.

> Sources (2)

Is this useful? thumbs up thumbs down

Share Download PDF

cost-genie-tables-only.png

Answered on one sense · thinking collapsed · Sources (2)

2 · Semantic layer

icks

What did we spend last month?

Reviewable

ChatMonitorBenchmark

What did we spend last month?

Thinking complete

I need to clarify what type of spending you're asking about, as "spend" could refer to different metrics in our data: 1. Marketing/Campaign Spend – Money spent on advertising campaigns across channels (Google Ads, Facebook, etc.) 2. Cost of Poor Quality (COPQ) – Costs associated with supplier quality issues and defects 3. Discount Amount – Discounts given on sales orders Could you please specify which type of spending you'd like to see for last month?

I need to clarify what type of spending you're asking about, as "spend" could refer to different metrics in our data:

1. Marketing/Campaign Spend – Money spent on advertising campaigns across channels (Google Ads, Facebook, etc.)

2. Cost of Poor Quality (COPQ) – Costs associated with supplier quality issues and defects

3. Discount Amount – Discounts given on sales orders

Could you please specify which type of spending you'd like to see for last month?

Is this useful?

cost-genie-semantic-layer.png

Asked · trace restates the reply · no sources

3 · Ontology + KG

What did we spend last month?

Reviewable

ChatMonitorBenchmarkConfigureShare

What did we spend last month?

Thinking complete

I want to make sure I use the right cost metric before pulling numbers.

Definitions used

The word spend is ambiguous here — it could mean marketing campaign ad spend, supplier cost of poor quality (COPQ), or sales order discounts given to customers. These are very different figures, so I have not defaulted to one — I need you to clarify which you mean before I pull numbers.

Which "spend" do you mean? That word maps to several different metrics in our data, and they'd give very different answers:

Marketing/campaign ad spend — actual media spend on marketing campaigns

Cost of Poor Quality (COPQ) — supplier quality-related costs

Sales order discounts — markdowns given to customers on orders

Let me know which one (or more) you'd like, and I'll also confirm "last month" against the most recent complete month available in that domain's data before reporting a figure.

Is this useful?

Ask your question...

About Sources Instructions Examples

How's our turnover?

What was our average ROAS by channel last month?

If Supplier X goes down tomorrow, which warehouses and parts are exposed?

What did we spend last month?

What's our marketing cost per lead by channel?

Which suppliers feed the parts sitting in the Pune warehouse right now?

Which plants were below the 70% OEE standard last week?

How many conversions did we get last month?

Which channels and segments does the Monsoon Festive campaign touch?

Show discount as a share of revenue

Which vehicle models does the Chennai plant actually build, and which lines build them?

About this agent

Name [dev.joyal.cs] Executive Analyst

Owner Joyal CS

Agent ID 01f19c980da81bf186b5711392228a4e

Warehouse Serverless Starter Warehouse

Tags

Thumbnail

cost-genie-ontology.png

Asked · "Definitions used" block · period rule stated

SIX CHECKPOINTS · ✓ CORRECT · ● PARTIAL, UNCOUNTED · ✗ WRONG OR ABSENT

CHECKPOINT	GOVERNED BY	1 · TABLES	2 · SEMANTIC	3 · + ONTOLOGY
"Spend" treated as ambiguous	ea:Cost disambiguatesTo – 4 senses	✗ answered on one sense	✓ asked	✓ asked
Senses enumerated	CampaignSpend · CostPerLead · COPQ · Discount	✗ none offered	● 3 of 4 — no cost per lead	● 3 of 4 — no cost per lead
No false claim of absence	COPQ on fact_supplier_quality; discount on fact_sales_order	✗ "the only 'spend' metric available in the data"	✓	✓

CHECKPOINT	GOVERNED BY	1 · TABLES	2 · SEMANTIC	3 · + ONTOLOGY
Period resolved and stated	per-domain max date; “last month” ≠ calendar assumption	✓ August 2026, justified as the latest month with records	✗ period never addressed	✓ committed to the most recent <i>complete</i> month per domain
Any figure published is at the right grain	Ertiga Family Festival spans Aug 1 – Oct 31	✗ full ₹1.2 Cr booked to August	✓ published no figure, and said why	✓ published no figure, and said why
Decision provenance visible	reader must be able to audit the vocabulary used	✗ thinking collapsed, no definitions	✗ thinking expanded — and it only restates the reply	✓ “Definitions used” block, named and expanded

VERDICT · QUESTION 1

The real gap is between Setup 1 and the other two, and **one instruction block buys it — not the ontology**. Setup 1 calculated nothing badly. It calculated the wrong thing with full confidence, then took away our reason to check by saying nothing else existed. One page of clarification text prevents that, and it is worth knowing before you buy an ontology.

Setups 2 and 3 make the same call, so the difference is whether we can audit it. We expanded Setup 2's thinking to find out: it is one bullet repeating its own answer. Setup 3's trace names the ambiguity, names the three candidate measures, and commits up front to a period rule. Both refuse to guess; only one shows us why. On a single question that hardly matters. Across a standing Benchmark set, it is the difference between a regression we can diagnose and one we can only watch.

Neither offered **cost per lead**, and the obvious explanation is wrong — our compile path carries all four meanings into the space instructions. Offering three is a fair reading of the word “spend”, since cost per lead is a ratio, not an amount spent.

03 Question 2 — a supplier blast radius

Q2 “If Supplier 001 goes down tomorrow, which warehouses and parts are exposed?”

This question asks the agent to follow a relationship, and there is no join it can use to do it. We gave `dim_supplier_contract` — one row per contracted (supplier, part) pair — to **none** of the three spaces. For Setups 1 and 2, the only column sourcing and inventory share is `PART_KEY`, sitting on two facts that count different things. So the obvious move is to link supplier to part through `fact_supplier_quality` — an *inspection* log. That is not a broken join. It is a narrower relationship wearing the right name: “supplies this part **and we have inspected it**”.

1 · Tables only

1 correction round · 8 query steps

3/9

INCOMPLETE, UNPRICED

2 · Semantic layer

6 correction rounds · ~17 query steps

3/9

INCOMPLETE, PRICED EXACTLY

3 · Ontology + KG

0 corrections · ~11 query steps

8/9

COMPLETE, DATED, UNPRICED

SETUP 1 SAID

“**6 warehouses and 3 parts** exposed, with 2 locations facing immediate stockout risk.” Everything about the three parts it found holds up. No money anywhere, no snapshot date.

SETUP 2 SAID

“**3 Parts Across 6 Warehouses Exposed** ... total inventory value **₹53.89 Lakh**” — per part, per warehouse and in total, reconciling to the rupee. Same three parts as Setup 1, after six repair rounds instead of one. It also queried part criticality twice and never reported it.

SETUP 3 SAID

“**4 Parts Across 7 Warehouses** ... 3 locations at critical or high stockout risk **as of September 4, 2026**.” The extra part is **Oil Seal** at Manesar Plant Store — HIGH risk, 1,521 units against 1,819 safety stock, 13 days of cover. It also turned up **Chennai RDC**, the seventh exposed warehouse, and classified every part B/C with subsystem. No rupee figure anywhere.

1. Tables only — 3 parts, 6 warehouses

New

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Discover

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie Agents

Alerts

Query History

SQL Warehouses

Data Engineering

Runs

Data Ingestion

Visual Data Prep

AI/ML

Playground

Agents

AI Gateway

Experiments

Features

Models

Serving

Search data, notebooks, recents, and more...

CTRL + P

agentsbricks-workspace

Chat

Monitor

Benchmark

If Supplier 001 goes down tomorrow, which warehouses and p

Reviewable

If Supplier 001 goes down tomorrow, which warehouses and parts are exposed?

Thinking complete

I'll analyze the exposure if Supplier 001 goes down by identifying which warehouses and parts depend on this supplier.

- Identify Supplier 001 details
 - Parts supplied by Supplier 001
 - Current inventory exposure by warehouse and part
- Let me fix those queries - I need to correct the join logic and column references.
 - Parts supplied by Supplier 001
 - Current inventory exposure by warehouse and part
 - Warehouse exposure summary
 - Part exposure summary
 - Days of supply by warehouse and part

Supplier 001 Exposure Analysis

If Supplier 001 goes down tomorrow, 6 warehouses and 3 parts would be exposed, with 2 locations facing immediate stockout risk.

Critical Exposure Points

Ahmedabad RDC faces the most severe risk with only 1 day of supply for Tie Rod End (PC-0066), classified as CRITICAL stockout risk. Gurgaon Plant Store has 7 days of supply for Oxygen Sensor (PC-0041), rated as HIGH risk.

Warehouse Exposure Summary

Warehouse	Location	Type	Parts Exposed	Min Days Supply	Critical Risk	High Risk	Medium Risk
Ahmedabad RDC	Ahmedabad, Gujarat	REGIONAL_DC	1	1	1	0	0
Gurgaon Plant Store	Gurgaon, Haryana	PLANT_STORE	2	7	0	1	0
Kolkata RDC	Kolkata, West Bengal	REGIONAL_DC	1	11	0	0	1
Manesar Plant Store	Manesar, Haryana	PLANT_STORE	2	20	0	0	0
Hyderabad RDC	Hyderabad, Telangana	REGIONAL_DC	1	29	0	0	0
Jaipur RDC	Jaipur, Rajasthan	REGIONAL_DC	2	31	0	0	0

Source: Warehouse exposure summary

Parts at Risk

Supplier 001 supplies 3 parts across the network:

- Tie Rod End (PC-0066): Class B critically, affects 4 warehouses with days of supply ranging from 1 to 29 days
- Oxygen Sensor (PC-0041): Class B critically, affects 4 warehouses with days of supply ranging from 7 to 31 days
- Exhaust Gasket (PC-0091): Class C critically, affects only Jaipur RDC with 2,521 days of supply

Both Class B parts have 1 high-risk location each, while the Class C part has adequate inventory.

Part Exposure Summary

Part Name	Part Code	Criticality	Warehouses	Min Days Supply	Avg Days Supply	High Risk Locations
Tie Rod End	PC-0066	B	4	1	19	1
Oxygen Sensor	PC-0041	B	4	7	17	1
Exhaust Gasket	PC-0091	C	1	2,521	2,521	0

Source: Part exposure summary

Warehouse-Part Inventory Status

The chart below shows days of supply for each warehouse-part combination. Locations to the left of the 7-day line require immediate attention:

Days of Supply by Warehouse-Part Combination

Source: Days of supply by warehouse and part

Recommended Actions

- Immediate: Expedite replenishment for Ahmedabad RDC (Tie Rod End) and Gurgaon Plant Store (Oxygen Sensor)
- Short-term: Monitor Kolkata RDC (11 days supply for Oxygen Sensor, medium risk)
- Strategic: Identify alternate suppliers for the two Class B parts, as Supplier 001 is rated A with low risk category but represents a single point of failure for these critical powertrain components

Sources (5)

Is this useful?

Share

Download PDF

Ask your question...

Always review the accuracy of responses.

Past chats

Today

If Supplier 001 goes down tomorrow, which warehouse...

Rank our suppliers by defect rate (PPM) — who is wor...

What did we spend last month?

There are no more conversations.

supplier-genie-tables-only.png

8 steps · 1 correction · Sources (5)

2 · Semantic layer — 3 parts, ₹53.89 L

+ New

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Discover

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie Agents

Alerts

Query History

SQL Warehouses

Data Engineering

Runs

Data Ingestion

Visual Data Prep

AI/ML

Playground

Agents

AI Gateway

Experiments

Features

Models

Serving

If Supplier 001 goes down tomorrow, which warehouses and p... Reviewable

Chat Monitor Benchmark

If Supplier 001 goes down tomorrow, which warehouses and parts are exposed?

Thinking complete

TB analyze which warehouses and parts would be exposed if Supplier 001 goes down. Let me investigate the current inventory and supplier relationships.

Parts and warehouses exposed to Supplier 001 disruption

Parts sourced from Supplier 001 with criticality

Warehouse exposure summary

Let me verify the supplier name format and fix the warehouse query

Verify Supplier 001 name format

Warehouse exposure summary fixed

Let me check what supplier names actually exist in the data

Executed query

Good! Supplier 001 exists. Now let me find which parts they supply and the inventory exposure

Parts supplied by Supplier 001

Inventory exposure by part and warehouse

I need to use a WHERE clause instead of a JOIN. Let me fix this

Inventory exposure by part and warehouse

Warehouse exposure summary

Part criticality details

Let me check the part dimension without the ISCURRENT filter

Part criticality details without ISCURRENT filter

Part-level exposure summary

Part-level exposure summary corrected

Critical and high risk locations

Now let me create a bar chart showing the part-level exposure

Supplier 001 Disruption: 3 Parts Across 6 Warehouses Exposed

If Supplier 001 goes down tomorrow, three parts would be affected across six warehouses, with a total inventory value of ₹53.89 Lakh.

Critical Exposure

Two locations face immediate risk:

Tie Rod End at Ahmedabad RDC: CRITICAL stockout risk with only 1 day of supply (15 units on hand vs. 64 safety stock). This location would run out almost immediately.

Oxygen Sensor at Gurgaon Plant Store: HIGH stockout risk with 7 days of supply (1,200 units on hand vs. 1,930 safety stock). This plant location is below safety stock and would face disruption within a week.

Inventory Exposure by Part and Warehouse

Part Name	Warehouse Name	Warehouse Type	Days of Supply	Qty on Hand	Safety Stock	Stock Value	Stockout Risk
Tie Rod End	Ahmedabad RDC	REGIONAL_DC	1	15.0	64.0	₹4.43K	CRITICAL
Oxygen Sensor	Gurgaon Plant Store	PLANT_STORE	7	1.2K	1.9K	₹48.00K	HIGH
Oxygen Sensor	Kolkata RDC	REGIONAL_DC	11	125.0	94.0	₹5.00K	MEDIUM
Oxygen Sensor	Manesar Plant Store	PLANT_STORE	20	1.4K	794.0	₹54.84K	LOW
Tie Rod End	Gurgaon Plant Store	PLANT_STORE	21	5.4K	2.8K	₹1.59M	LOW
Tie Rod End	Manesar Plant Store	PLANT_STORE	26	10.7K	4.7K	₹3.17M	LOW
Tie Rod End	Hyderabad RDC	REGIONAL_DC	29	198.0	86.0	₹58.41K	LOW
Oxygen Sensor	Jaipur RDC	REGIONAL_DC	31	415.0	165.0	₹16.60K	LOW

Source: Inventory exposure by part and warehouse

Parts Supplied by Supplier 001

Tie Rod End represents the largest exposure at ₹48.2M in inventory value, followed by Exhaust Gasket (₹4.44 Lakh) and Oxygen Sensor (₹1.24 Lakh).

Total Stock Value by Part

Part Name	Total Stock Value
Tie Rod End	₹4.82M
Exhaust Gasket	₹4.44L
Oxygen Sensor	₹1.24L

Source: Part-level exposure summary corrected

Warehouse Impact

Plant stores face the highest exposure:

Manesar Plant Store: 2 parts exposed (Tie Rod End, Oxygen Sensor), ₹32.26 Lakh inventory value

Gurgaon Plant Store: 2 parts exposed (Tie Rod End, Oxygen Sensor), ₹16.34 Lakh inventory value

Regional distribution centers have lower but still material exposure across Jaipur, Hyderabad, Kolkata, and Ahmedabad.

Warehouse Exposure Summary

Warehouse Name	Warehouse Type	Parts Exposed	Total Stock Value	Total Qty on Hand
Manesar Plant Store	PLANT_STORE	2	₹3.23M	12.1K
Gurgaon Plant Store	PLANT_STORE	2	₹1.63M	6.6K
Jaipur RDC	REGIONAL_DC	2	₹461.00K	1.2K
Hyderabad RDC	REGIONAL_DC	1	₹58.41K	198.0
Kolkata RDC	REGIONAL_DC	1	₹5.00K	125.0
Ahmedabad RDC	REGIONAL_DC	1	₹4.43K	15.0

Source: Warehouse exposure summary

Recommended Actions

Immediate: Expedite replenishment for Tie Rod End at Ahmedabad RDC (1 day cover) and Oxygen Sensor at Gurgaon Plant Store (7 days cover)

Short-term: Identify alternate suppliers for these three parts to reduce single-source dependency

Monitor: Track consumption rates at the two plant stores, which hold the bulk of exposed inventory

> Sources (3)

Is this useful? Like Dislike Feedback

Share Download PDF

Ask your question...

Feedback Upvote Downvote

3 · Ontology + KG — 4 parts, 7 warehouses

databricks

Search data, notebooks, recents, and more...

CTRL + P

agentbricks-workspace

ConfigureShare

New

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Discover

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie Agents

Alerts

Query History

SQL Warehouses

Data Engineering

Runs

Data Ingestion

Visual Data Prep

AI/ML

Playground

Agents

AI Gateway

Experiments

Features

Models

Serving

If Supplier 001 goes down tomorrow, which warehouses and parts are exposed?

ChatMonitorBenchmark

Thinking complete

- I'll investigate which warehouses and parts would be exposed if Supplier 001 goes down. Let me start by identifying what Supplier 001 supplies and where those parts are stocked.
 - Identify Supplier 001
 - Find Supplier 001 in knowledge graph
- Now let me find which parts Supplier 001 supplies and where those parts are currently stocked
 - Find parts supplied by Supplier 001
 - Parts supplied by Supplier 001 with criticality
- Now let me check the current inventory status for these parts across all warehouses
 - Current inventory status for Supplier 001 parts
 - Get latest snapshot date
 - Summary of exposure by part
 - Summary of exposure by warehouse
 - Detailed exposure by part and warehouse with risk
- Risk distribution by warehouse
- Inventory levels by part

Supplier 001 Exposure: 4 Parts Across 7 Warehouses

If Supplier 001 goes down, 4 parts across 7 warehouses would be exposed, with 3 locations at critical or high stockout risk as of September 4, 2026.

Critical Exposure Points

Immediate Risk (Critical/High):

- Ahmedabad RDC: Tie Rod End at CRITICAL risk — only 15 units on hand vs 64 safety stock (1 day of supply)
- Gurgaon Plant Store: Oxygen Sensor at HIGH risk — 1,200 units vs 1,920 safety stock (7 days of supply)
- Manesar Plant Store: Oil Seal at HIGH risk — 1,521 units vs 1,819 safety stock (13 days of supply)

Show code

Supplier 001 Exposure: Parts and Warehouses at Risk

Part	Criticality	Warehouse	Type	Risk Level	Qty on Hand	Safety Stock	Days of Supply
Tie Rod End	B	Ahmedabad RDC	REGIONAL_DC	Critical	15.0	64.0	1
Oxygen Sensor	B	Gurgaon Plant Store	PLANT_STORE	High	1.2K	1.9K	7
Oil Seal	C	Manesar Plant Store	PLANT_STORE	High	1.5K	1.8K	13
Oxygen Sensor	B	Jaipur RDC	REGIONAL_DC	Low	415.0	165.0	31
Oxygen Sensor	B	Manesar Plant Store	PLANT_STORE	Low	1.4K	794.0	20
Tie Rod End	B	Gurgaon Plant Store	PLANT_STORE	Low	5.4K	2.8K	21
Tie Rod End	B	Hyderabad RDC	REGIONAL_DC	Low	198.0	86.0	29
Tie Rod End	B	Manesar Plant Store	PLANT_STORE	Low	10.7K	4.7K	26

Source: Detailed exposure by part and warehouse with risk

Parts Supplied by Supplier 001

Supplier 001 (Tier-1, Gurgaon) supplies 4 parts:

- Oxygen Sensor (B-class, Powertrain) — 4 warehouses affected, 1 high-risk location
- Tie Rod End (B-class, Powertrain) — 4 warehouses affected, 1 critical-risk location
- Oil Seal (C-class, Chassis) — 4 warehouses affected, 1 high-risk location
- Exhaust Gasket (C-class, Chassis) — 1 warehouse affected, low risk

Show code

Risk Distribution by Warehouse

Warehouse	Risk Level
Ahmedabad RDC	Critical
Gurgaon Plant Store	High
Manesar Plant Store	High
Chennai RDC	Low
Hyderabad RDC	Low
Jaipur RDC	Low
Kolkata RDC	Low

Source: Risk distribution by warehouse

Warehouse Impact Summary

Plant Stores (highest exposure):

- Gurgaon Plant Store: 3 parts affected, 16,966 units on hand, 1 high-risk part
- Manesar Plant Store: 3 parts affected, 13,641 units on hand, 1 high-risk part

Regional DCs:

- Ahmedabad RDC: 1 part (Tie Rod End) at CRITICAL risk with only 15 units
- Jaipur RDC: 3 parts affected, all at low risk
- Chennai, Hyderabad, Kolkata RDCs: 1 part each, medium to low risk

Show code

Inventory Levels vs Safety Stock by Part

Part	Inventory Level	Safety Stock
Oxygen Sensor	~1.2K	~1.9K
Tie Rod End	~5.4K	~2.8K
Exhaust Gasket	~1.5K	~1.8K
Oil Seal	~1.5K	~1.8K

Source: Inventory levels by part

Recommended Actions

- Immediate: Expedite Tie Rod End replenishment to Ahmedabad RDC (250 units in transit but critically low stock)
- Short-term: Boost Oxygen Sensor inventory at Gurgaon Plant Store and Oil Seal at Manesar Plant Store to above safety stock levels
- Strategic: Identify alternate suppliers for the two B-class parts (Oxygen Sensor, Tie Rod End) given their criticality and current exposure

Sources (5)

Is this useful?

ShareDownload PDF

Ask your question...

Always review the accuracy of responses.

supplier-genie-ontology.png

~11 steps · 0 corrections · Sources (5)

Setup 2 is our strongest argument for the ontology layer, and it makes that argument by being *good*. It is not sloppy. It is governed, its measures are certified, its maths adds up, and it repeats. It is also missing a quarter of the answer, and every one of those strengths makes the gap harder to see.

NINE CHECKPOINTS · ✓ CORRECT · ● PARTIAL, UNCOUNTED · ✗ WRONG OR ABSENT

CHECKPOINT	GOVERNED BY	1 · TABLES	2 · SEMANTIC	3 · + ONTOLOGY
Supplier entity resolved	Supplier 001 · Tier-1 · Gurgaon	✓ resolved	✓ after a name probe and two failed queries	✓ one <code>kg_find_node</code>
Part set complete	4 – Oxygen Sensor, Tie Rod End, Oil Seal, Exhaust Gasket (<i>contract-derived</i>)	✗ 3 — Oil Seal absent	✗ 3 — Oil Seal absent	✓ 4
Warehouse set complete	7, including Chennai RDC	✗ 6	✗ 6	✓ 7
Every immediate-risk location found	3 – Ahmedabad CRITICAL, Gurgaon HIGH, Manesar HIGH	✗ 2 of 3, reported as the total	✗ 2 of 3, reported as the total	✓ 3
Risk bands over the closed enum	<code>mfg:RiskLevel = LOW / MEDIUM / HIGH / CRITICAL</code>	✓ all four bands	✓ all four bands	✓ all four bands
Snapshot date stated	<code>point-in-time grain; is_current_snapshot</code>	✗ undated	✗ undated	✓ “as of September 4, 2026”
Monetary exposure quantified	<code>MEASURE(total_stock_valuation)</code> on the current snapshot	✗ days of supply and units only	✓ ₹53.89 L, reconciling per part and per warehouse	✗ units only, no currency anywhere
Part criticality classified	B / B / C / C, with subsystem	✓ Class B, B, C	✗ queried twice, never reported	✓ B/C plus Powertrain / Chassis
Reached the answer without rewriting failed queries	determinism and cost	🟡 1 correction round, 8 steps	✗ 6 correction rounds, ~17 steps	✓ none, ~11 steps

VERDICT · QUESTION 2

The two lower setups tie at 3 of 9, and how they got there matters more than the tie. Setup 1 classified the parts and never priced them. Setup 2 priced them to the rupee and never classified them. Neither found the fourth part, the seventh warehouse, or the third at-risk location, and neither told us what date it was looking at. Both presented their totals as totals.

This is not the model being smarter, and the matching sources prove it. Setups 2 and 3 got the same 25 tables, and neither holds `dim_supplier_contract`, so **no prompt will ever get Setup 2 to the fourth part**. That ceiling is built in. The fix is a deployment, not a prompt.

Setup 3's single loss is the fair counterweight, and it flips the ranking on the one thing an executive is most likely to ask next. It gave us the complete picture and no money. Setup 2 gave us exact money over an incomplete picture. Of those two faults, the missing rupee column is the one we can actually see.

04 Why the graph found the fourth part

This is the whole report in one comparison, and it has nothing to do with how good the model is.

Two tables can tell you who supplies a part. The **contract register** lists every part a supplier has agreed to supply. That is the real answer, and it holds 60 rows. The **inspection log** lists every part somebody has checked the quality of. It looks like the same thing and holds 45 rows. Neither table's name warns you about the difference.

The lower two setups never had the contract register, so the inspection log was their only route. They took it and lost a quarter of the answer — **3 of 4 parts, 75%**, which is exactly the ratio the inspection route yields across the whole

catalog. We build the graph from the contract register on purpose, because we already hit this same bug once, when we built the graph the wrong way round:

```
-- src/ontology/edge_sources.sql    (hand-written, runs before materialize_kg)
--
-- Contract-derived, not event-derived. The previous source for supplierSuppliesPart
-- was fact_supplier_quality – an inspection-grain fact – which silently narrowed the
-- relation to "supplies this part AND we have inspected it": 45 of 60 contracted
-- parts. Parts never inspected (e.g. Blower Motor P0071, Headrest Guide P0088) had a
-- preferred supplier on contract but no edge in the graph, so traversals reported
-- them as unsourced.
CREATE OR REPLACE VIEW ontology.part_supplier_sourcing AS
  SELECT s.SUPPLIER_KEY, p.PART_KEY, c.is_preferred
  FROM      supply_chain_analytics.dim_supplier_contract c
  JOIN      dim.dim_supplier s ON s.SUPPLIER_ID = c.supplier_id
  JOIN      dim.dim_part      p ON p.PART_ID      = c.part_id;

-- Guard: every contract row must resolve to both surrogate keys, or fail the run.
-- assert_kg_integrity's empty_rel check only fires at zero edges, so it would not
-- notice this relation quietly shrinking again – exactly how the original defect survived.
```

That guard exists because this same relationship already shrank once and nobody noticed. Setups 1 and 2 reproduce that exact bug live in this study. The check is the difference between a bug we fix once and a bug that returns on the next dimension reload.

So the ontology does not make the agent cleverer. It means somebody wrote down, once, which of two similar-looking tables is the real one — and now the agent cannot pick the other. On Question 2 that was two function calls: `kg_find_node('Supplier 001')`, then `kg_neighbors('Supplier', key, 'supplierSuppliesPart')`. Following the graph replaced all the hunting.

Three layers, three kinds of knowledge

LAYER	IN ONE LINE	THE QUESTION IT ANSWERS FOR THE AGENT	WHAT GOES WRONG WITHOUT IT
Semantic layer	A dictionary of numbers	"When someone says <i>inventory value</i> , which column do I add up, and what am I not allowed to add it across?"	Every answer works out the maths from scratch. Ask twice, get two different numbers, and nothing tells you a choice was made.
Ontology	A dictionary of words	"Does <i>depot</i> mean warehouse? Does <i>spend</i> have more than one meaning? Is <i>at risk</i> three bands or four?"	The agent picks one meaning of your word and answers that perfectly. You never find out it chose.
Knowledge graph	A map of connections	"Which supplier feeds this part? Which warehouses hold it? Can I walk from one to the other without guessing a join?"	The agent joins whichever two tables share a column. The answer is right about the rows it found, and missing the ones it never found.

We maintain one of the three by hand. `exec_analyst.ttl` is 743 lines; from it `generate.py` writes the graph DDL, the two lookup functions, seven data assertions, the glossary tags, the disambiguation context and the space config — eight artifacts, each with a *do not edit by hand* header, and the build fails if any is older than the TTL.

Brief · Part 1 of 2 · evaluation of six captured Genie Agent sessions · two questions × three grounding setups on `gold_dev_analytics`, agent `[dev_joyal_cs]` Executive Analyst, Serverless Starter Warehouse, inventory snapshot 2026-09-04.

Part 2 — `docs/genie-agent-grounding-brief-part2.html` — carries \$05 the side-by-side scorecard and \$06 the recommendation. Condensed from `docs/genie-agent-grounding.html`, which carries all six session transcripts, the full build walkthrough for the three layers, the ranked failure modes

and the verification queries. Structural claims are grounded in `src/ontology/edge_sources.sql`, `src/ontology/exec_analyst.ttl` and `src/metric_views/`.